

EXPERIMENT NO. 7 - MongoDB

Name of Student	Gunjan Chandnani
Class Roll No	D15A_06
D.O.P.	
D.O.S.	
Sign and Grade	

AIM: To study CRUD operations in MongoDB

PROBLEM STATEMENT:

1. Create a new database to storage student details of IT dept(Name, Roll no, class name) and perform the following on the database
 - a. Insert one student details
 - b. Insert at once multiple student details
 - c. Display student for a particular class
 - d. Display students of specific roll no in a class
 - e. Change the roll no of a student
 - f. Delete entries of particular student
2. Create a set of RESTful endpoints using Node.js, Express, and Mongoose for handling student data operations. The endpoints should support:
 - a. Retrieve a list of all students.
 - b. Retrieve details of an individual student by ID.
 - c. Add a new student to the database.
 - d. Update details of an existing student by ID.
 - e. Delete a student from the database by ID.Connect the server to MongoDB using Mongoose, and store student data with attributes: name, age, and grade.

OUTPUT:

1. Create a database to store student details of IT Department

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.4.2
C:\Users\gunja>mongosh
Current Mongosh Log ID: 67fd178b7002c64bbab71235
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.4.2
Using MongoDB:  8.0.0
Using Mongosh:  2.4.2
mongosh 2.5.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2025-04-14T19:41:00.097+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2025-04-14T19:41:00.098+05:30: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> use IT_Dept
switched to db IT_Dept
IT_Dept> db.students.insertOne({
...   name: "Gunjan Chandnani",
...   roll_no: 101,
...   class_name: "TYIT-A"
... })
...
{
  acknowledged: true,
  insertedId: ObjectId('67fd17b87002c64bbab71236')
}
```

a) Insert one Student Detail

```
test> use IT_Dept
switched to db IT_Dept
IT_Dept> db.students.insertOne({
...   name: "Gunjan Chandnani",
...   roll_no: 101,
...   class_name: "TYIT-A"
... })
...
{
  acknowledged: true,
  insertedId: ObjectId('67fd17b87002c64bbab71236')
}
```

b) Insert at once multiple Student Details

```
IT_Dept> db.students.insertMany([
...   { name: "Aditi Mehta", roll_no: 102, class_name: "TYIT-A" },
...   { name: "Rohan Sharma", roll_no: 103, class_name: "TYIT-B" },
...   { name: "Priya Desai", roll_no: 104, class_name: "TYIT-A" }
... ])
...
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67fd17c47002c64bbab71237'),
    '1': ObjectId('67fd17c47002c64bbab71238'),
    '2': ObjectId('67fd17c47002c64bbab71239')
  }
}
```

c) Display Students for a Particular Class

```
IT_Dept> db.students.find({ class_name: "TYIT-A" }).pretty()
[
  {
    _id: ObjectId('67fd17b87002c64bbab71236'),
    name: 'Gunjan Chandnani',
    roll_no: 101,
    class_name: 'TYIT-A'
  },
  {
    _id: ObjectId('67fd17c47002c64bbab71237'),
    name: 'Aditi Mehta',
    roll_no: 102,
    class_name: 'TYIT-A'
  },
  {
    _id: ObjectId('67fd17c47002c64bbab71239'),
    name: 'Priya Desai',
    roll_no: 104,
    class_name: 'TYIT-A'
  }
]
```

d) Display Student with Specific Roll No in a Class

```
IT_Dept> db.students.find({ class_name: "TYIT-A", roll_no: 102 }).pretty()
[
  {
    _id: ObjectId('67fd17c47002c64bbab71237'),
    name: 'Aditi Mehta',
    roll_no: 102,
    class_name: 'TYIT-A'
  }
]
```

6. Change the Roll No of a Student:

```
IT_Dept> db.students.updateOne(
...   { name: "Gunjan Chandnani" },
...   { $set: { roll_no: 111 } }
... )
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

7. Delete a Student's Entry:

```
IT_Dept> db.students.deleteOne({ name: "Aditi Mehta" })
{ acknowledged: true, deletedCount: 1 }
```

8. View All Students

```
IT_Dept> db.students.find().pretty()
[
  {
    _id: ObjectId('67fd17b87002c64bbab71236'),
    name: 'Gunjan Chandnani',
    roll_no: 111,
    class_name: 'TYIT-A'
  },
  {
    _id: ObjectId('67fd17c47002c64bbab71238'),
    name: 'Rohan Sharma',
    roll_no: 103,
    class_name: 'TYIT-B'
  },
  {
    _id: ObjectId('67fd17c47002c64bbab71239'),
    name: 'Priya Desai',
    roll_no: 104,
    class_name: 'TYIT-A'
  }
]
IT_Dept> |
```

CONCLUSION

In this experiment, we successfully performed CRUD operations in **MongoDB** and implemented a **RESTful API** using **Node.js**, **Express**, and **Mongoose**. We learned how to create, read, update, and delete student records both via **MongoDB shell commands** and **API endpoints**.

